

AI

คู่มือการใช้งาน

ai_web_scrpping

ระบบรวบรวมและวิเคราะห์ข้อมูลเชิงพื้นที่

อสังหาริมทรัพย์ ถนน จุดสำคัญ และบริบทพื้นที่

เวอร์ชัน 1.0 • จัดทำวันที่ 09/06/2026

Python Data Pipeline • Playwright • Google Maps API • AWS S3

คู่มือนี้จัดทำสำหรับผู้ทั่วไป นักวิเคราะห์ข้อมูล และนักพัฒนา



บทที่ 1	ภาพรวมโปรเจกต์	3
บทที่ 2	โครงสร้างโฟลเดอร์	4
บทที่ 3	สิ่งที่ต้องเตรียมก่อนใช้งาน	5
บทที่ 4	วิธีรัน Pipeline หลัก (main.py)	7
บทที่ 5	คำสั่งย่อยที่ใช้งานบ่อย	9
บทที่ 6	Data Flow และสถาปัตยกรรมระบบ	11
บทที่ 7	Output และ Data Contract	13
บทที่ 8	Quality Gate	15
บทที่ 9	Troubleshooting	16
บทที่ 10	Best Practices	18
บทที่ 11	ภาคผนวก	19

บทที่ 1

ภาพรวมโปรเจกต์

โปรเจกต์ **ai_web_scrpping** คือระบบ Python Data Pipeline ที่ออกแบบมาเพื่อรวบรวม ทำความสะอาด และวิเคราะห์ข้อมูลเชิงพื้นที่ที่สำคัญ (Landmarks) ร้านอาหาร และโครงข่ายถนนในประเทศไทยแบบครบวงจร โดยดึงข้อมูลจากแหล่งต่าง ๆ ทั้ง Google Maps, OpenStreetMap และหน่วยงานรัฐ มาประมวลผลให้อยู่ในรูปแบบที่พร้อมใช้งาน

ข้อมูลมาจากไหน?

แหล่งข้อมูล	ประเภทข้อมูล	หมายเหตุ
Google Maps	จุดสำคัญ / ร้านอาหาร / สิ่งอำนวยความสะดวก	Scraping ผ่าน Playwright
OpenStreetMap / Road API	โครงข่ายถนน / POI บางส่วน	ผ่าน run_roads_only.py
Weather API (TMD)	สถิติสภาพอากาศ	สำหรับบริบทพื้นที่
DOPA (กรมการปกครอง)	ข้อมูลประชากรรายอายุ	สำหรับการวิเคราะห์พื้นที่
GISTDA	ข้อมูลพื้นที่เสี่ยงน้ำท่วม	ต้องใช้ GISTDA_API_KEY

ผลลัพธ์ (Output) ให้อะไรได้?

ข้อมูลที่ได้ผ่าน pipeline แล้วจะถูกจัดเก็บในรูปแบบ CSV/JSON ใน **data/processed/** ซึ่งพร้อมนำไปใช้กับเครื่องมือวิเคราะห์ได้ทันที เช่น:

Dashboard และ BI Tools — Power BI, Tableau, Looker Studio รองรับ CSV โดยตรง

Location Intelligence — วิเคราะห์ทำเลที่ตั้ง โครงข่ายถนน และการเข้าถึงสถานที่สำคัญผ่าน QGIS

วิเคราะห์ความหนาแน่นจุดสำคัญ — สรุปการกระจายตัวของ Landmarks และ สิ่งอำนวยความสะดวก รายเขต

สรุปประจำบท

- ระบบดึงข้อมูลเชิงพื้นที่และจุดสำคัญพร้อมกันผ่าน Python Scraper
- Output พร้อมใช้กับ BI Tools, QGIS, และ Dashboard
- วิเคราะห์เส้นทางและการเข้าถึง (Road Intelligence) ได้แบบละเอียด

บทที่ 2

โครงสร้างโฟลเดอร์

โปรเจกต์ติดตั้งอยู่ที่ C:\ai_web_scrpping โดยมีการจัดระเบียบโฟลเดอร์ดังนี้:

```
C:\ai_web_scrpping\  
├── main.py          # Entry point: Full pipeline (Interactive)  
├── run_zones_only.py  # Entry point: Zone pipeline  
├── run_landmarks_only.py  # Landmarks เท่านั้น  
├── run_restaurants_only.py  # Restaurants เท่านั้น  
├── run_roads_only.py   # Roads เท่านั้น  
├── run_amenities_only.py  # Amenities เท่านั้น  
├── run_google_road_enrichment.py  
├── run_property_road_access.py  
├── run_property_road_access_qgis_export.py  
├── run_road_intelligence.py  
├── .env             # API Keys (อยู่ใน .gitignore แล้ว)  
├── venv/            # Python Virtual Environment  
├── scrapers/        # Web scrapers  
├── utils/           # Utility modules  
├── data/  
│   ├── raw/         # ข้อมูลดิบจาก scrapers  
│   └── processed/    # ข้อมูลที่ผ่านการ clean แล้ว  
├── tests/           # Unit tests (35 tests สำหรับ road modules)  
├── mockups/         # ไฟล์ mockups สำหรับออกแบบ  
├── output/          # ไฟล์ output อื่นๆ  
├── tmp/             # ไฟล์ชั่วคราว  
├── qgis/            # QGIS project files  
└── docs/           # เอกสารประกอบ
```

คำอธิบายโฟลเดอร์หลัก

โฟลเดอร์	บทบาท	หมายเหตุสำคัญ
scrapers/	เก็บ Python scraper แต่ละตัว สำหรับแต่ละแหล่งข้อมูล	ห้ามรัน scraper โดยตรงจาก folder นี้ — ใช้ entry point เสมอ
utils/	โมดูล utility เช่น data cleaner, merger, zone analyzer	ถูกเรียกโดย main.py และ run_*.py โดยอัตโนมัติ
data/raw/	เก็บข้อมูลดิบที่ดึงมาได้ (JSON, CSV, เป็นต้น)	Google Maps sync จะเขียนทับ landmarks_raw.json
data/processed/	ข้อมูลที่ผ่านการ clean/merge แล้ว พร้อมใช้งาน	นี่คือ canonical output ที่ควรใช้ต่อ

โฟลเดอร์	บทบาท	หมายเหตุสำคัญ
tests/	Unit tests สำหรับตรวจสอบความถูกต้องของข้อมูล	รันด้วย pytest
qgis/	ไฟล์โปรเจกต์ QGIS สำหรับ visualization พื้นที่	ต้องติดตั้ง QGIS แยกต่างหาก
docs/	เอกสารประกอบโปรเจกต์	รวมถึงคู่มือนี้
venv/	Python virtual environment เวอร์ชัน 3.13	Activate ก่อนรันทุกครั้ง

⚠ ข้อควรระวัง: การรัน scraper โดยตรง

อย่ารัน scraper ใน scrapers/ โดยตรงจาก repo root หรือ folder อื่น เพราะ scraper หลายตัวใช้ relative path แบบ ../data/... ซึ่งจะผิดพลาดหากไม่ได้รันจาก entry point ที่ถูกต้อง

วิธีที่ถูกต้องคือรัน python main.py หรือ python run_zones_only.py เสมอ

สรุปประจำบท

- Entry point หลักมีเพียง main.py และ run_zones_only.py
- ข้อมูลดิบอยู่ที่ data/raw/ — processed output อยู่ที่ data/processed/
- อย่ารัน scraper ใน scrapers/ โดยตรง
- venv ต้อง activate ก่อนทุกครั้ง

บทที่ 3

สิ่งที่ต้องเตรียมก่อนใช้งาน

ก่อนรัน pipeline ทุกครั้ง ต้องตรวจสอบว่าสิ่งต่อไปนี้พร้อมใช้งานแล้ว:

ขั้นตอนที่ 1 — เปิด Terminal ที่ตำแหน่ง root

เปิด PowerShell หรือ Command Prompt แล้ว cd ไปที่ root ของโปรเจกต์:

```
cd C:\ai_web_scrppng
```

➤ **สำคัญ: ต้องรันจาก root เสมอ**

ต้องรันคำสั่งทั้งหมดจาก C:\ai_web_scrppng เท่านั้น หากรันจาก folder อื่น path จะผิดพลาดและ scraper จะหาไฟล์ไม่เจอ

ขั้นตอนที่ 2 — Activate Virtual Environment

รันคำสั่งต่อไปนี้เพื่อเปิดใช้งาน Python virtual environment:

```
# สำหรับ PowerShell
.\env\Scripts\Activate.ps1
```

```
# สำหรับ Command Prompt (cmd)
.\env\Scripts\activate.bat
```

เมื่อ activate สำเร็จ จะเห็น (venv) ปรากฏที่หน้า prompt เช่น: (venv) PS C:\ai_web_scrppng>

i Python เวอร์ชัน

โปรเจกต์นี้ต้องการ Python 3.13 ตามที่กำหนดใน venv/pyenvn.cfg หากใช้ Python เวอร์ชันอื่น อาจเกิดปัญหา dependency ได้

ขั้นตอนที่ 3 — ติดตั้ง Playwright Browser

Scraper หลายตัวในโปรเจกต์ใช้ Playwright เพื่อเปิด browser อัตโนมัติ หากยังไม่เคยติดตั้ง หรือ binary หายไป ให้รันคำสั่งนี้:

```
python -m playwright install chromium
```

Playwright จะดาวน์โหลด Chromium browser binary มาติดตั้ง ให้เวลาสักครู่

ขั้นตอนที่ 4 — ตั้งค่าไฟล์ .env

สร้างหรือแก้ไขไฟล์ .env ที่ root ของโปรเจกต์ โดยกำหนด API Key ที่จำเป็น:

```
# ตัวอย่างโครงสร้าง .env (ห้ามใส่ค่าจริงใน document นี้)
BOT_API_KEY=your_bank_of_thailand_api_key_here
```

```
GOOGLE_MAPS_API_KEY=your_google_maps_api_key_here
AWS_ACCESS_KEY_ID=your_aws_access_key_id
AWS_SECRET_ACCESS_KEY=your_aws_secret_access_key
AWS_DEFAULT_REGION=ap-southeast-1
S3_BUCKET_NAME=your-target-bucket-name
```

สรุป API Key ที่จำเป็น

API Key	ใช้ทำอะไร	จำเป็นสำหรับ
BOT_API_KEY	ดึงข้อมูลสินเชื่อธนาคาร จาก Bank of Thailand	main.py (bank loan scraper)
GISTDA_API_KEY	ดึงข้อมูลพื้นที่เสี่ยงน้ำท่วม จาก GISTDA	scrapers/flood_risk.py
GOOGLE_MAPS_API_KEY	Reverse geocode ชื่อถนน ที่ไม่มีชื่อบน Google Maps	utils/google_road_enrichment.py
AWS Credentials	อัปโหลดข้อมูลขึ้น AWS S3	เฉพาะเมื่อใช้ S3 upload ใน main.py (แก่ S3 Bucket ในโค้ด)

⚠ อย่า Commit ไฟล์ .env

ไฟล์ .env มี API Key ที่มีความลับ ห้าม commit ลง Git repository โดยเด็ดขาด ตรวจสอบให้แน่ใจว่า .env อยู่ใน .gitignore แล้ว

ขั้นตอนที่ 5 — ตรวจสอบ Network และ API Access

ก่อนรัน pipeline ควรตรวจสอบว่า:

- เครื่องมีการเชื่อมต่ออินเทอร์เน็ตที่เสถียร
- Google Maps API Key ยังมี Quota เหลือ (ตรวจสอบที่ Google Cloud Console)
- BOT_API_KEY ยังมีผลบังคับใช้
- หาก S3 upload เปิดใช้งาน ตรวจสอบว่า bucket name ถูกต้องและสิทธิ์ถูกต้อง

สรุปประจำบท

- รันทุกอย่างจาก C:\ai_web_scrpping เสมอ
- Activate venv ก่อนรันทุกครั้ง: .\env\Scripts\Activate.ps1
- ต้องมีไฟล์ .env ที่ถูกต้องพร้อม API Keys ก่อนรัน
- ติดตั้ง Playwright: python -m playwright install chromium
- อย่า commit ไฟล์ .env ลง Git

บทที่ 4

วิธีรัน Pipeline หลัก (main.py)

คำสั่ง **python main.py** เป็นระบบเมนูโต้ตอบ (Interactive Menu) ซึ่งมีตัวเลือกให้คุณรัน pipeline ทั้งหมด หรือเลือกเฉพาะส่วนที่ต้องการได้

คำสั่งรัน

```
# ตรวจสอบว่า activate venv แล้ว
(venv) PS C:\ai_web_scrpping> python main.py
```

โหมดการทำงานของ main.py

เมื่อเรียกใช้คำสั่ง จะปรากฏเมนูให้เลือก 4 โหมด:

🔗 Web Scraping Pipeline — เมนูหลัก	
[1]	รันทุกอย่าง (Full Pipeline)
[2]	ดึงเฉพาะ Landmarks (OSM + Google Maps Sync)
[3]	ดึงเฉพาะ Restaurants
[4]	เลือกเองทีละรายการ (Custom)
[q]	ออกจากโปรแกรม

หากเลือก [1] **รันทุกอย่าง (Full Pipeline)** ระบบจะทำงานตามลำดับดังนี้:

ขั้นตอน	ชื่องาน	Output ที่ได้	หมายเหตุ
1	Scrape Landmarks & Sync	data/raw/landmarks_raw.json	OSM + Google Maps Sync (Playwright)
2	Scrape Restaurants	data/raw/restaurants_raw.json	OSM + Google Maps
3	Clean & Merge Data	data/processed/*.csv	ลบ duplicates และรวมข้อมูล
4	Quality Gate	— (ตรวจสอบ)	Pipeline หยุดถ้า gate fail
5	S3 Upload	S3 bucket	แก้ hardcoded S3 Bucket ใน main.py

⚠️ ข้อควรระวังสำหรับ main.py

S3 Bucket Placeholder: ค่าเริ่มต้น bucket ชื่อ "your-target-bucket-name" ต้องแก้ไขใน .env ให้เป็น bucket จริงก่อนใช้งาน production มิฉะนั้น S3 upload จะ fail

Google Maps Sync: ทุกครั้งที่รัน main.py ไฟล์ landmarks_raw.json จะถูกเขียนทับด้วยข้อมูลใหม่จาก Google Maps

ตัวอย่าง Output ที่คาดหวัง

เมื่อ pipeline รันสำเร็จ จะเห็น log ประมาณนี้:

```
[INFO] Landmarks sync complete: 850 POIs
[INFO] Restaurants sync complete: 1,200 records
[INFO] Cleaning and merging data...
[INFO] Quality gate: PASSED
[INFO] Uploading to S3... Done
[SUCCESS] Pipeline completed successfully
```

✓ เคล็ดลับ: ตรวจสอบ Log

ระหว่างรัน ให้สังเกต log message ที่หน้า terminal สำหรับ [ERROR] หรือ [WARNING] หากพบให้หยุดและแก้ไขปัญหาก่อนรัน pipeline ต่อ

ดู Troubleshooting สำหรับปัญหาที่พบบ่อย

สรุปประจำบท

- รัน full pipeline ด้วย: python main.py
- เน้นกระบวนการดึงข้อมูล Landmarks และ ร้านอาหาร
- Quality gate จะหยุด pipeline ถ้าข้อมูลไม่ผ่านเกณฑ์

บทที่ 6

คำสั่งย่อยที่ใช้งานบ่อย

นอกจาก main.py แล้ว ยังมีคำสั่งย่อยสำหรับรันส่วนเฉพาะเจาะจง ซึ่งมีประโยชน์เมื่อต้องการ debug หรืออัปเดตเฉพาะส่วน:

python run_landmarks_only.py	
ใช้เมื่อ	ต้องการดึง / อัปเดตข้อมูล Landmarks และ POI เท่านั้น
Input	Google Maps API Key ใน .env
Output	data/raw/landmarks_raw.json data/processed/landmarks_clean.csv
ข้อควรระวัง	เขียนทับ landmarks_raw.json ทุกครั้งที่รัน

python run_roads_only.py	
ใช้เมื่อ	ต้องการดึง / อัปเดตข้อมูลโครงข่ายถนน
Input	Network connection (OpenStreetMap API)
Output	data/raw/road_network.json
ข้อควรระวัง	อาจใช้เวลาานถ้าพื้นที่ใหญ่

python run_google_road_enrichment.py	
ใช้เมื่อ	เพิ่มข้อมูล road details จาก Google Maps (road type, speed limit, etc.)
Input	data/raw/road_network.json Google Maps API Key
Output	data/processed/roads_enriched.csv
ข้อควรระวัง	ใช้ Google API Quota มาก

python run_amenities_only.py

ใช้เมื่อ	ดึงข้อมูล amenities เช่น โรงเรียน โรงพยาบาล ห้างสรรพสินค้า
Input	Playwright
Output	data/raw/amenities_raw.json data/processed/amenities_clean.csv
ข้อควรระวัง	ใช้ Google Maps (ไม่ใช่ API Key)

python run_restaurants_only.py

ใช้เมื่อ	ดึงข้อมูลร้านอาหาร (มีเมนูเลือกจังหวัด)
Input	Playwright
Output	data/raw/restaurants_raw.json data/processed/restaurants_clean.csv
ข้อควรระวัง	มี PROVINCE env var สำหรับ CI

python run_road_intelligence.py

ใช้เมื่อ	สร้าง feature และ summary ของถนน
Input	data/processed/roads.csv
Output	roads_features.csv roads_summary.csv
ข้อควรระวัง	ใช้วิเคราะห์ road density

สรุปประจำบท

- ใช้คำสั่งย่อยเมื่อต้องการอัปเดตข้อมูลส่วนใดส่วนหนึ่งโดยเฉพาะ
- ตรวจสอบ input files ที่จำเป็นก่อนรันคำสั่งย่อยทุกครั้ง
- คำสั่งที่ใช้ Google API จะใช้ Quota ควรตรวจสอบก่อนรัน

บทที่ 5

Data Flow และสถาปัตยกรรมระบบ

ภาพรวมการไหลของข้อมูลตั้งแต่ต้นทางจนถึงปลายทาง แสดงให้เห็นว่าข้อมูลแต่ละประเภทเดินทางผ่านกระบวนการใดบ้างก่อนเป็น output สุดท้าย:

ภาพรวม Data Flow หลัก

แหล่งข้อมูลภายนอก	Scrapers	data/raw/	Cleaners	data/processed/
Google Maps OSM TMD DOPA GISTDA	Python Playwright Scrapers API Clients	landmarks_raw.json road_network.json restaurants_raw.json	Data Cleaners Merge Utils (utils/)	landmarks_clean.csv roads_enriched.csv restaurants_clean.csv

แผนภาพแสดง data flow จากต้นทางถึงปลายทาง

Data Flow แยกตามประเภทข้อมูล

Landmarks / POI	Google Maps API → run_landmarks_only.py → landmarks_raw.json → data cleaner → landmarks_clean.csv → Dashboard
Road Intelligence	OSM / Google Maps → run_roads_only.py → road_network.json → run_google_road_enrichment.py → roads_enriched.csv → QGIS
Restaurants	Google Maps → run_restaurants_only.py → restaurants_raw.json → data cleaner → restaurants_clean.csv

สรุปประจำบท

- ข้อมูลเชิงพื้นที่และถนนผ่าน scrapers → raw → processed
- Road intelligence มี flow แยกและ export เป็น QGIS ได้
- Quality gate เป็นด่านสุดท้ายก่อนนำไปใช้

บทที่ 7

Output และ Data Contract

ส่วนนี้อธิบายไฟล์ output สำคัญของ pipeline บทบาท และ field/column ที่ควรรู้ เพื่อนำข้อมูลไปใช้ต่อได้อย่างถูกต้อง

ไฟล์ Processed Output (Canonical)

✓ ใช้ Processed CSV เป็น Canonical Output

ไฟล์ใน data/processed/ คือ output ที่ผ่านการ clean แล้วและควรใช้เป็นแหล่งข้อมูลหลัก ไฟล์ใน data/raw/ เป็น intermediate artifact ที่อาจเปลี่ยนรูปแบบได้โดยไม่แจ้ง

data/processed/landmarks_clean.csv	
บทบาท	ข้อมูลจุดสำคัญ (POI) ที่ clean แล้ว
สร้างโดย	Landmark scraper + Google Maps sync
ใช้ต่อที่	QGIS, Dashboard
Field สำคัญ	id, name, lat, lng, category, province, layer (Layer 1=สำคัญมาก, 2=สำคัญ, 3=ทั่วไป)

data/processed/roads*.csv	
บทบาท	ข้อมูลโครงข่ายถนน และ Road Features
สร้างโดย	utils/road_intelligence.py
ใช้ต่อที่	QGIS, Dashboard, Data Science Models
Field สำคัญ	มีหลายไฟล์ เช่น roads.csv, roads_enriched.csv, roads_features.csv, road_density_by_zone.csv

data/processed/restaurants_clean.csv	
บทบาท	ข้อมูลร้านอาหารที่ clean แล้ว
สร้างโดย	Restaurant scraper
ใช้ต่อที่	Dashboard, QGIS
Field สำคัญ	id, name, lat, lng, rating, user_ratings_total

ไฟล์ Raw Output (Intermediate)

ไฟล์เหล่านี้เป็น intermediate artifacts ไม่ควรนำไปใช้เป็น canonical data โดยตรง:

ไฟล์	บทบาท	หมายเหตุ
data/raw/landmarks_raw.json	ข้อมูล landmarks ดิบก่อน clean	ถูกเขียนทับทุกครั้งที่ sync
data/raw/road_network.json	โครงข่ายถนนดิบจาก OSM	ใช้เป็น input ของ road enrichment
data/raw/restaurants_raw.json	ข้อมูลร้านอาหารดิบ	Input ของ cleaner

สรุปประจำบท
• ใช้ data/processed/*.csv เป็น canonical output เสมอ • data/raw/ เป็น intermediate — อาจเปลี่ยนรูปแบบได้

บทที่ 8

Quality Gate

Quality Gate คือกระบวนการตรวจสอบคุณภาพข้อมูลอัตโนมัติที่รันก่อน pipeline จะถือว่า สำเร็จ หาก gate fail pipeline จะหยุดและแสดง error โดยไม่อัปโหลดข้อมูลขึ้น S3

⚠ Quality Gate Fail = Pipeline หยุดทันที

หาก quality gate ตรวจพบปัญหา pipeline จะหยุดและไม่อัปโหลดข้อมูลขึ้น S3 ต้องแก้ไขปัญหาก่อนแล้วรัน pipeline ใหม่

เกณฑ์ Quality Gate

หัวข้อตรวจ	เงื่อนไข	จะ Fail เมื่อ
ครอบคลุมจังหวัด (Landmarks)	Landmarks ต้องมีข้อมูลครบตามเป้าหมาย	จังหวัดใดจังหวัดหนึ่งในเป้าหมายไม่มี landmark เลย
Layer Coverage (Landmarks)	ต้องมี landmark ใน Layer 1, 2 และ 3	Layer ใด Layer หนึ่งว่างเปล่า
ข้อมูลร้านอาหาร	ข้อมูลร้านอาหารต้องมีมากกว่า 0	ไม่มีข้อมูลร้านอาหารใน CSV เลย
Row Count Minimum	แต่ละ output CSV ต้องมี row ไม่น้อยกว่าเกณฑ์ขั้นต่ำ	CSV ว่างหรือ row count ต่ำกว่าเกณฑ์ขั้นต่ำ

วิธีตรวจสอบ Quality Gate Result

ดูผลได้จาก log ที่หน้า terminal:

```
# ตัวอย่าง log เมื่อผ่าน
[INFO] Quality Gate: landmarks OK
[INFO] Quality Gate: restaurants OK
[SUCCESS] Quality Gate: PASSED - proceeding to upload

# ตัวอย่าง log เมื่อ fail
[ERROR] Quality Gate: landmarks - missing layers
[ERROR] Quality Gate: FAILED - pipeline stopped
```

✓ วิธีแก้เมื่อ Quality Gate Fail

ตรวจสอบ log เพื่อดูว่า gate ไหน fail

หาก landmarks ไม่ครบ — รัน run_landmarks_only.py ใหม่ หรือตรวจสอบ Google API Key

หาก row count ต่ำ — อาจมีปัญหา network หรือ scraper ถูก block

สรุปประจำบท

- Quality gate ตรวจ 4 เงื่อนไขก่อน pipeline สำเร็จ
- Gate fail = pipeline หยุด ไม่มี S3 upload
- ตรวจสอบ log เพื่อหาสาเหตุที่ gate fail
- แก้ปัญหาแล้วรัน pipeline ใหม่เสมอ

บทที่ 10

Troubleshooting

ตารางด้านล่างรวบรวมปัญหาที่พบบ่อยพร้อมสาเหตุและวิธีแก้ไข:

อาการที่พบ	สาเหตุ	วิธีแก้ไข
Playwright ไม่สามารถเปิด browser ได้ "Browser not found" หรือ "Executable missing"	Browser binary หาย หรือ ยังไม่เคยติดตั้ง	รัน: python -m playwright install chromium แล้วลองใหม่
Google API ตอบกลับ "REQUEST_DENIED" หรือ "INVALID_KEY"	API Key ใน .env ผิด หมดอายุ หรือ Maps API ยังไม่เปิดใช้	ตรวจสอบ GOOGLE_MAPS_API_KEY ใน .env และ Google Cloud Console ว่า Maps & Places API enabled
bank_loans_clean.csv ว้าง หรือ row count = 0	BOT_API_KEY ไม่มีใน .env หรือ Key หมดอายุ	ตรวจสอบและอัปเดต BOT_API_KEY ใน .env แล้วรันใหม่
S3 upload fail: "NoSuchBucket" หรือ "Access Denied"	S3 Bucket ใน main.py เป็น placeholder หรือ AWS credentials ไม่ถูกต้อง	แก้ตัวแปร S3 Bucket ใน main.py ให้เป็น bucket จริง และตรวจสอบ AWS credentials
Output CSV มี row น้อยกว่าที่คาดหวัง หรือ field ว้างมาก	Scraper อาจถูก block หรือ network timeout	ตรวจสอบ internet connection รอแล้วลองใหม่ หรือดู log error
Quality Gate: FAILED "only X/10 provinces"	ข้อมูลบางจังหวัดดึงไม่ได้ อาจเกิดจาก scraper error หรือ API rate limit	ดู log ว่าจังหวัดใดหายไป ตรวจสอบ network/API แล้วรันใหม่
Path error / FileNotFoundError เมื่อรัน scraper โดยตรง	รัน scraper จาก folder ที่ผิด (ไม่ได้รันจาก repo root)	cd C:\ai_web_scrpping แล้วรัน python main.py หรือ รัน entry point เสมอ
Google Maps API: OVER_QUERY_LIMIT หรือ quota exceeded	ใช้ API calls เกิน quota รายวัน	รอถึงวันถัดไป หรืออัปเกรด Google Cloud plan ขยาย quota
ImportError หรือ ModuleNotFoundError เมื่อ import library	venv ยังไม่ได้ activate หรือ dependency ยังไม่ติดตั้ง	ตรวจสอบว่า activate venv แล้ว รัน: pip install -r requirements.txt
venv activation fail: "cannot be loaded because running scripts is disabled"	PowerShell Execution Policy บล็อก script	รัน PowerShell เป็น Admin แล้วรัน: Set-ExecutionPolicy RemoteSigned แล้ว activate ใหม่

สรุปประจำบท

- ปัญหา Playwright: python -m playwright install chromium

- ปัญหา API: ตรวจสอบ .env และ key ใน cloud console
- ปัญหา path: รันจาก C:\ai_web_scrpping เสมอ
- ปัญหา venv: ตรวจสอบ Set-ExecutionPolicy สำหรับ PowerShell

บทที่ 11

Best Practices

i หมายเหตุ

รันจาก Repo Root เสมอ

รันคำสั่งทุกคำสั่งจาก C:\ai_web_scrpping เสมอ อย่าเปลี่ยน directory ระหว่างการรัน เพราะ scraper ใช้ relative path ที่อ้างอิงจาก root

⚠ ข้อควรระวัง

อย่าลบ Raw Files โดยไม่จำเป็น

ไฟล์ใน data/raw/ เป็น intermediate artifacts ที่ใช้ใน pipeline อย่าลบหรือแก้ไขโดยไม่จำเป็น

✗ ข้อผิดพลาด

อย่า Commit ไฟล์ .env

ไฟล์ .env มี API Key และ credentials ที่มีความลับ ต้องอยู่ใน .gitignore เสมอ ห้าม commit ลง Git โดยเด็ดขาด

✓ เคล็ดลับ

ใช้ Processed CSV เป็น Canonical Output

นำเฉพาะไฟล์ใน data/processed/ ไปใช้กับ Dashboard หรือ BI Tools ไฟล์ใน data/raw/ เป็น intermediate ที่อาจเปลี่ยนรูปแบบได้โดยไม่แจ้ง

✓ เคล็ดลับ

ตรวจสอบ Row Count ก่อนส่งต่อ Dashboard

ก่อนนำ CSV ไปใช้กับ Dashboard หรือ BI ควรตรวจสอบ row count และ quality gate ว่าผ่านเรียบร้อยแล้ว เพื่อป้องกัน dashboard แสดงข้อมูลผิดพลาด

⚠ ข้อควรระวัง

Raw Files คือ Intermediate ไม่ใช่ Contract

อย่า hardcode schema ของ data/raw/ ใน application อื่น เพราะรูปแบบ อาจเปลี่ยนตามการอัปเดต scraper ใช้เฉพาะ data/processed/ เป็น contract

Security Checklist

- .env อยู่ใน .gitignore
- ไม่ hardcode API Key ใน source code
- ตรวจสอบ AWS IAM permission ให้มีสิทธิ์เฉพาะที่จำเป็น
- เปลี่ยน API Key ทันทีถ้าสงสัยว่า leak
- จำกัดสิทธิ์ S3 bucket ตาม principle of least privilege

สรุปประจำบท

- รันจาก repo root เสมอ — ห้ามรัน scraper โดยตรง
- ใช้ data/processed/ เป็น canonical output
- ป้องกัน .env จาก Git commit
- ตรวจสอบ row count และ quality gate ก่อนส่งต่อ Dashboard

บทที่ 12

ภาคผนวก

คำศัพท์และคำย่อที่ใช้

คำศัพท์	ความหมาย
POI (Point of Interest)	จุดสนใจหรือจุดสำคัญบนแผนที่ เช่น โรงพยาบาล โรงเรียน ห้างสรรพสินค้า
Landmark	จุดสำคัญที่ใช้เป็น anchor ในการ define zone และบริบทพื้นที่
Layer 1 / 2 / 3	การจัดกลุ่ม landmark ตามความสำคัญ Layer 1 = สำคัญมาก, 2 = สำคัญ, 3 = ทั่วไป
Canonical Output	ข้อมูล output หลักที่ถือว่าถูกต้องและพร้อมใช้งาน (data/processed/)
Quality Gate	กระบวนการตรวจสอบคุณภาพข้อมูลอัตโนมัติก่อน pipeline สำเร็จ
Zone Profile	โปรไฟล์ที่สรุปลักษณะและศักยภาพของพื้นที่แต่ละ zone
Scraper	โปรแกรม Python ที่ดึงข้อมูลจากเว็บไซต์หรือ API โดยอัตโนมัติ
Playwright	library Python สำหรับควบคุม browser อัตโนมัติ ใช้ในการ scrape เว็บ
Pipeline	ลำดับกระบวนการประมวลผลข้อมูลตั้งแต่ต้นทางจนถึงปลายทาง
BOT_API_KEY	API Key สำหรับเข้าถึงข้อมูลจากธนาคารแห่งประเทศไทย
venv	Virtual Environment ของ Python เพื่อแยก dependencies ของโปรเจกต์
S3 / AWS S3	บริการจัดเก็บไฟล์บน cloud ของ Amazon Web Services
Road Network	โครงข่ายถนนในรูปแบบ graph data ที่ใช้วิเคราะห์การเข้าถึง
Intermediate Artifact	ไฟล์ที่สร้างระหว่าง pipeline ใช้เป็น input ของขั้นตอนถัดไป

Checklist ก่อนรัน Production

☐	อยู่ที่ C:\ai_web_scrpping แล้ว (cd มาถูก)
☐	Activate venv แล้ว (.venv\Scripts\Activate.ps1)
☐	ไฟล์ .env มีค่าครบ: BOT_API_KEY, GISTDA_API_KEY, GOOGLE_MAPS_API_KEY
☐	S3 Bucket ใน main.py L127 ถูกแก้เป็น bucket จริง (ถ้าใช้ S3)
☐	Google Maps API Quota เหลือเพียงพอ
☐	Playwright Chromium ติดตั้งแล้ว

☐	Internet connection เสถียร
☐	ไม่มี background process รัน scraper อยู่แล้ว (หลีกเลี่ยง double run)

Checklist หลังรัน Pipeline

☐	Log แสดง "[SUCCESS] Pipeline completed successfully"
☐	Quality Gate: PASSED ปรากฏใน log
☐	ตรวจสอบ row count ของ landmarks_clean.csv (ควร > 500 rows)
☐	ตรวจสอบ S3 bucket ว่าไฟล์ upload ขึ้นไปแล้ว (ถ้าใช้ S3)
☐	แจ้ง stakeholder ว่ามีข้อมูลใหม่พร้อมใช้งาน

ตัวอย่างโครงสร้างไฟล์ .env

สร้างไฟล์ชื่อ .env ที่ C:\ai_web_scrpping\ โดยใช้ template ต่อไปนี้ (แทนที่ค่า your_... ด้วยค่าจริง):

```
# =====  
# .env template - ai_web_scrpping project  
# อย่า commit ไฟล์นี้ลง Git !  
# =====  
  
# ----- Google Maps API -----  
GOOGLE_MAPS_API_KEY=your_google_maps_api_key_here  
  
# ----- Bank of Thailand API -----  
BOT_API_KEY=your_bank_of_thailand_api_key_here  
  
# ----- AWS S3 Credentials -----  
AWS_ACCESS_KEY_ID=your_aws_access_key_id  
AWS_SECRET_ACCESS_KEY=your_aws_secret_access_key  
AWS_DEFAULT_REGION=ap-southeast-1  
# (S3 Bucket Name ถูกตั้งค่าใน main.py)  
  
# ----- GISTDA API (Flood Risk) -----  
GISTDA_API_KEY=your_gistda_api_key  
  
# ----- Optional Settings -----  
# LOG_LEVEL=INFO  
# SCRAPE_DELAY_SECONDS=2
```

เอกสารนี้จัดทำขึ้นเพื่อประกอบการใช้งานโปรเจกต์ ai_web_scrpping เท่านั้น หากมีข้อสงสัยหรือพบปัญหาที่ไม่ได้อยู่ในคู่มือนี้
กรุณาติดต่อทีมพัฒนา หรือตรวจสอบ README.md ในโปรเจกต์